



APS Lógica da Computação
João Vitor Heinke Siqueira

Introdução ao Projeto FutLang

Construindo uma Linguagem de Programação Temática com VM



Objetivo do Projeto

Desenvolver uma linguagem de programação de alto nível com suporte a variáveis, estruturas condicionais e de repetição, voltada para execução em uma máquina virtual personalizada.



Integração com VM

A linguagem deve gerar um código de saída em Assembly específico para uma VM com registradores, sensores e capacidade de Turing-completude.



Padrões Utilizados

Estrutura formal baseada em EBNF para definição da gramática, análise léxica com Flex, análise sintática com Bison e geração de código intermediário em C++.

Motivação e Tema da Linguagem

Futebol como Metáfora Lúdica de Programação



Criatividade como Ferramenta Didática

A personalização da linguagem com palavras-chave como 'cartao', 'gol', 'tocar' e 'prorrogaçao' reforça o vínculo com o tema e estimula o raciocínio analítico.



Motivação

Um tema divertido, comum e simples de ser entendido.

Características da Linguagem FutLang

Funcionalidades e Estrutura Lógica



Gramática em EBNF

A linguagem é formalmente definida em EBNF, descrevendo sintaxe para declarações, comandos, expressões e ações com clareza e extensibilidade.



Palavras-chave Temáticas

Comandos como 'jogador', 'cartao', 'prorrogaçao', e 'gol' trazem coerência com o tema futebolístico, mantendo funcionalidade típica de linguagens estruturadas.



Estruturas Fundamentais

Inclui suporte a variáveis, operadores aritméticos e lógicos, condicionais, laços 'for' e 'while', além de blocos aninhados.



Sensores Read-only e Ações

Sensores como 'placar', 'tempo' e 'estadio' são variáveis de leitura; ações como 'chutar' e 'marcar' representam interações com a VM.

Especificação Formal da Linguagem

Definição Sintática via EBNF



Estrutura Central

A linguagem é composta por declarações e comandos que podem ser combinados em blocos, utilizando gramática recursiva e bem estruturada.



Declarações e Comandos

Suporta declaração de variáveis com ou sem inicialização, além de comandos como atribuição, ações, blocos e estruturas de controle.



Condicionais e Laços

Usa construções como 'cartao' para if/else, 'prorrogacao' para while, e 'acrescimos' para for, todas representadas na gramática EBNF.



Operadores e Precedência

A gramática define expressões com operadores relacionais, lógicos e aritméticos com precedência explícita e regras unárias.

Análise Léxica com Flex

Identificação de Tokens e Reconhecimento de Padrões



Definição de Tokens

O analisador léxico reconhece palavras-chave, identificadores, operadores, literais numéricos e booleanos com expressões regulares específicas.



Ignorando Espaços e Comentários

Espaços em branco e comentários (`//` e `/* */`) são descartados na análise, conforme definido nas regras léxicas.



Integração com Bison

Flex gera um scanner em C++ que envia tokens ao parser Bison para análise sintática subsequente.



Tratamento de Erros

Símbolos inválidos geram mensagens específicas durante a análise léxica, permitindo depuração eficiente.

Análise Sintática com Bison

Construção da Árvore Sintática e Geração de Código

- **Regras Gramaticais:** Bison utiliza as regras definidas no arquivo .y para estruturar a linguagem, correspondendo diretamente à EBNF utilizada.
 - **Construção da AST:** Cada produção sintática ativa ações semânticas em C++, que geram elementos da árvore sintática abstrata (AST).
 - **Integração com Flex:** Recebe os tokens do scanner Flex, permitindo análise bottom-up de expressões e comandos complexos.
 - **Geração de Código Intermediário:** O parser produz instruções específicas em Assembly da VM-alvo, que serão escritas em um arquivo de saída.
-

Integração com a Máquina Virtual

Conectando a Linguagem ao Ambiente de Execução

- **Arquitetura da VM:** A máquina virtual implementada possui registradores, memória, sensores (read-only) e uma pilha, garantindo Turing-completude.
- **Assembly Customizado:** O código gerado pela FutLang é traduzido para instruções compatíveis com a VM-alvo, que executa os programas em tempo simulado.
- **Suporte a Sensores:** Variáveis como 'placar', 'tempo' e 'estadio' são acessadas como sensores de leitura, influenciando decisões de execução.
- **Execução e Simulação:** A VM interpreta instruções como saltos, operações aritméticas, push/pop de pilha, condicionais e interações via ações.

Exemplos de Código FutLang

Condicionais, Loops e Ações em Contexto

- **Condicional com 'cartao'**: `if (cartao) ... else (substituicao)` – estrutura que permite bifurcação de lógica com tema esportivo.
- **Laço com 'prorrogacao'**: `while (condição)` – utilizado para repetir comandos enquanto a condição for verdadeira, como uma prorrogação.
- **Loop com 'acrescimos'**: `for (inicialização; condição; incremento)` – estrutura típica de repetição com sintaxe adaptada ao tema.
- **Ações como 'chutar' e 'marcar'**: Comandos para interações explícitas com a VM, acionando efeitos como marcar pontos ou mover estados.

Execução e Resultados

Testes com Programas em FutLang na VM

- **Testes com Loops:** Execução do programa 'ex_loops.mwasm' demonstrou controle de tempo e ações repetitivas simulando acréscimos em uma partida.
- **Condicional 'cartao' em Ação:** Execução de 'ex_if_else.mwasm' demonstrou a bifurcação do fluxo com e sem substituição, afetando diretamente o estado final da VM.
- **Interpretação pela VM:** A máquina virtual interpretou corretamente instruções como TIME, POWER e BEEEEEP, validando a geração do Assembly.
- **Estado Final da Simulação:** Sensores como TEMP, WEIGHT e TIME apresentaram valores consistentes após execução, comprovando a lógica da linguagem.

Curiosidades e Decisões Técnicas

Inovações e Escolhas Estratégicas no Projeto



Design Temático Único

A inspiração no futebol guiou a escolha de palavras-chave e comandos, tornando a linguagem mais engajante e divertida.



Balanceamento entre Criatividade e Técnica

Apesar do tom lúdico, a linguagem mantém precisão formal com EBNF, análise léxica e sintática robustas.



Geração de Assembly Específico

Cada construção da linguagem gera instruções para uma VM customizada, testando diretamente o modelo computacional subjacente.



Expansibilidade Planejada

A estrutura modular da linguagem permite inclusão futura de funções, arrays ou novos sensores com impacto mínimo.

Conclusão do Projeto FutLang

Resultados, Aprendizados e Próximos Passos

- **Objetivos Alcançados:** A linguagem desenvolvida apresenta todas as estruturas fundamentais e integra-se com sucesso a uma máquina virtual funcional.
- **Aprendizado Prático:** Os estudantes vivenciaram todas as etapas de criação de uma linguagem: definição gramatical, análise léxica, parsing e geração de código.
- **Experiência Multidisciplinar:** O projeto conectou conceitos de linguagens formais, compiladores, lógica e arquitetura de sistemas.
- **Possíveis Extensões:** A FutLang pode ser expandida com novas funcionalidades como funções, escopos ou integração com outras VMs.